UNIDADE 2

Reactividade e Ciclo de Vida dos compoñentes en VUE

Obxectivos da unidade

- Comprender o concepto de *reactividade* en Vue.
- Distinguir entre **compoñentes funcionais** e **con estado.**
- Empregar ***ref()*** e ***reactive()*** para manexar datos.
- Usar propiedades ***computed*** e o hook ***onMounted()***.

A.1. Reactividade en VUE

En *Vue*, a reactividade significa que o *framework* é capaz de **detectar automaticamente** cando **cambian os datos** e **actualizar a interface de usuario (DOM)** sen recargar a páxina.

Neste senso, en *Vue* temos tres funcións básicas ***ref, reactive*** e ***computed*** para traballar a reactividade. Todas serven para **crear variables reactivas,** e dicir, cambian co seu valor e *Vue* actualiza automaticamente a interface.

Todas fan o mesmo pero **no se usan igual** ni serven exactamente para o mesmo.

Vémolo con un exemplo cada caso.

1. ref()

- ***ref***() manexa variables primitivas coma **números, cadeas de texto, booleanos ou calquera valor simple incluíndo *arrays* simples** que non formen parte de obxectos máis complexos.
- Accedemos ao valor ca expresión ***.value***

```
import { ref } from 'vue';

const contador = ref(0);
contador.value++;          // Vue actualiza o DOM automáticamente
```

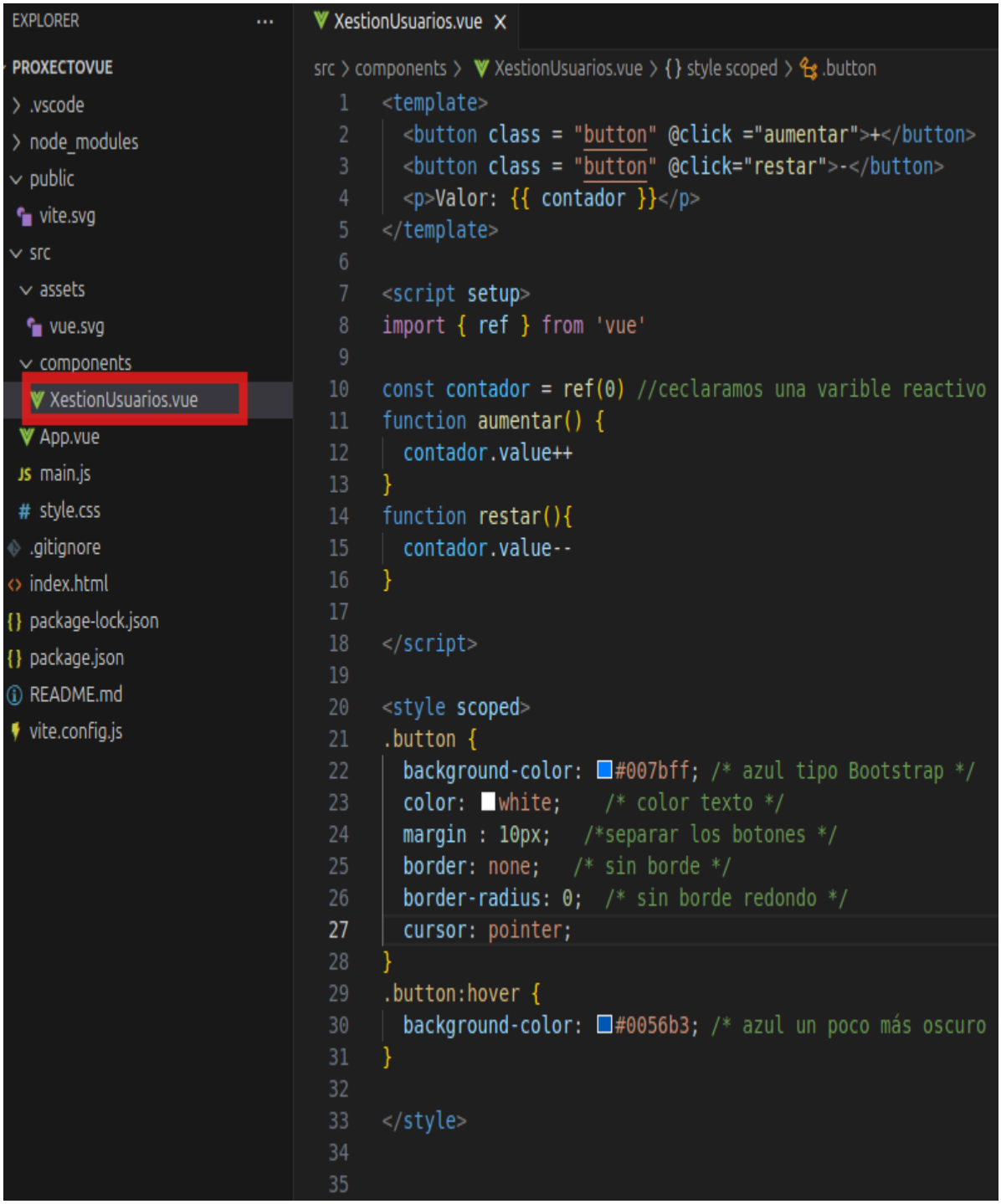
Vemos un exemplo. Imos crear un compoñente ***XestionUsuarios.vue,*** onde faremos as probas iniciais eliminando os compoñentes que houbera (ex. *HelloWorld.vue*).

Ante de nada dicir, que ó principio este compoñente será onde practicaremos estes conceptos básicos de *Vue* deixando tamén aparcado a aplicación de estilos web. Sen embargo hai que indicar os estilos dende a zona de *style* impórtanse así:

Opción 1: con ***class*** recomendado por ser reutilizable	**Opción 2**: con ***id*** no CSS
`<div class="menu">Inicio</div>`	`<div id="menu">Inicio</div>`

`<style scoped>`
`.nome {`
`color: blue;`
`font-weight: bold;`
`}`
`</style>`
	`<style scoped>`
	`#nome {`
	`color: blue;`
	`font-weight: bold;`
	`}`
	`</style>`

Volvendo a este compoñentes que estamos a crear. A estrutura sería algo así:



Por outra banda, como vimos na unidade anterior, ***App.vue*** é a raíz onde arranca a aplicación por iso imos facer algún cambio nel para que se poida ver o novo compoñente.

Así que facemos as modificacións que se mostran na seguinte imaxe.

App.vue

src > App.vue > {} style > {} @media (max-width: 768px) > .app-container

```vue
<template>
  <main class="container">
    <XestionUsuarios />
  </main>
</template>

<script setup>
import XestionUsuarios from './components/XestionUsuarios.vue'
</script>

<style>
.container {
  width: 70vw;         /* 70% del ancho de la pantalla */
  max-width: 1200px;   /* opcional: límite máximo para pantallas grandes */
  height: 100vh;       /* 100% del alto de la pantalla */
  margin: 0 auto;      /* centrado horizontal */
  padding: 1rem;
  box-sizing: border-box;
}

/* Responsive para pantallas pequeñas */
@media (max-width: 768px) {
  .app-container {
    width: 95vw;     /* casi todo el ancho */
    padding: 0.5rem;
  }
}
</style>
```

Non se nos esqueza de arrincar o servizo **Vue,** se non estivese iniciado.

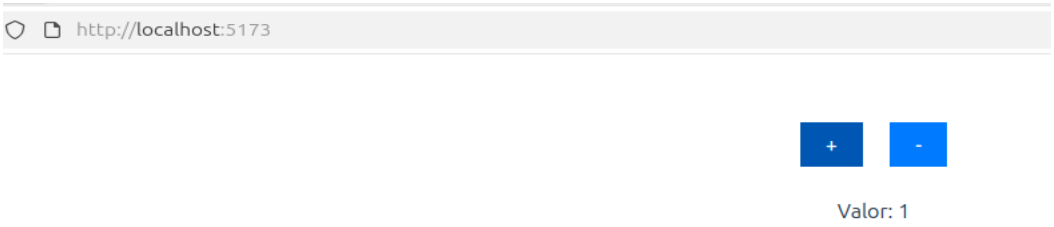
```
usuario@cursovue:~/proxectovue$ npm run dev

> proxectovue@0.0.0 dev
> vite


  VITE v7.1.3  ready in 3743 ms

  ➜  Local:   http://localhost:5173/
```

O resultado:



Se pulsamos nos botóns irá cambiando o valor numérico executando o código situado na zona de *script* que mostra posteriormente na páxina sen que esta se recargue xa que vai chamando as funcións *aumentar* e *restar* asociadas aos botóns mediante ***@click***

2. reactive

Usase para **valores máis complexos** como *obxectos* ou *estruturas de datos.*

Engadamos no compoñente ***XestionUsuarios.vue*** anterior os seguintes códigos marcados en vermello.

```
<template>
  <button class = "button" @click ="aumentar">+</button>
  <button class = "button" @click="restar">-</button>
  <p>Valor: {{ contador }}</p>

  <h5> Xogador </h5>
  <p> Nome: {{ xogador.nome }}</p>
  <p> Puntuación: {{ xogador.puntuacion }}</p>

</template>

<script setup>
import { ref, reactive  } from 'vue'

const contador = ref(0) //declaramos una varible reactivo ref

const xogador = reactive  //declaramos como reactive
({
  nome: "Xose",
  puntuacion: 0

})

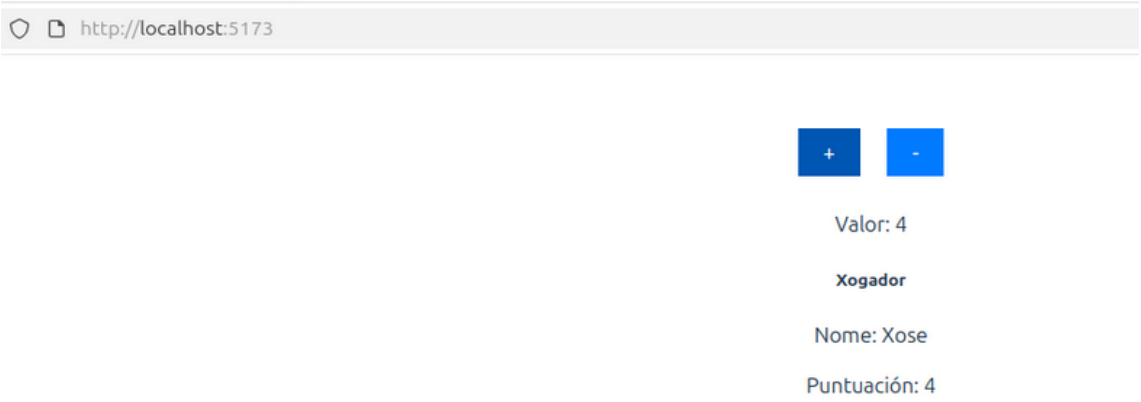
function aumentar() {
  contador.value++
  xogador.puntuacion++
}
function restar(){
  contador.value--
  xogador.puntuacion--
}

</script>
```

Podemos observar que temos unha **estrutura de datos** chamada *xogador* que imos a utilizar para manter os seus cambios con ***reactive*** tal como explicamos antes. Pero o máis importante é o seguinte que é outra diferencia máis entre **ref** y **reactive.**

Mentres a ***ref*** accedemos ao valor co termo ***.value,*** *contador.value)*, a estrutura de datos **co nome da mesma** e o valor correspondente, neste caso *xogador.puntuacion.*

Resultado:



3. computed

As propiedades ***computed*** serven para **calcular valores derivados**, xeralmente **a partires de outros datos reactivos** de forma automática. É importante ter en conta que **non almacena datos** como poden facer *reactive* ou *ref,* senón que **devolven un resultado que depende de outros**.

Solo cando estes cambian *computed* actúa. Entre as súas vantaxes están a *reutilización* de código e mellora do rendemento xa que actúan só cando fai falla.

Neste caso observamos que a función computada que se chama ***estadoxogador*** depende dunha variable reactive.

```
<template>
  <button class = "button" @click ="aumentar">+</button>
  <button class = "button" @click="restar">-</button>
  <p>Valor: {{ contador }}</p>

  <h5> Xogador </h5>
  <p> Nome: {{ xogador.nome }}</p>
  <p> Puntuación: {{ xogador.puntuacion }}</p>
  <p> {{ estadoxogador }}</p>

</template>

<script setup>
import { ref, reactive, computed  } from 'vue'

const contador = ref(0) //declaramos una varible reactivo ref

const xogador = reactive  //declaramos como reactive
({
  nome: "Xose",
  puntuacion: 0

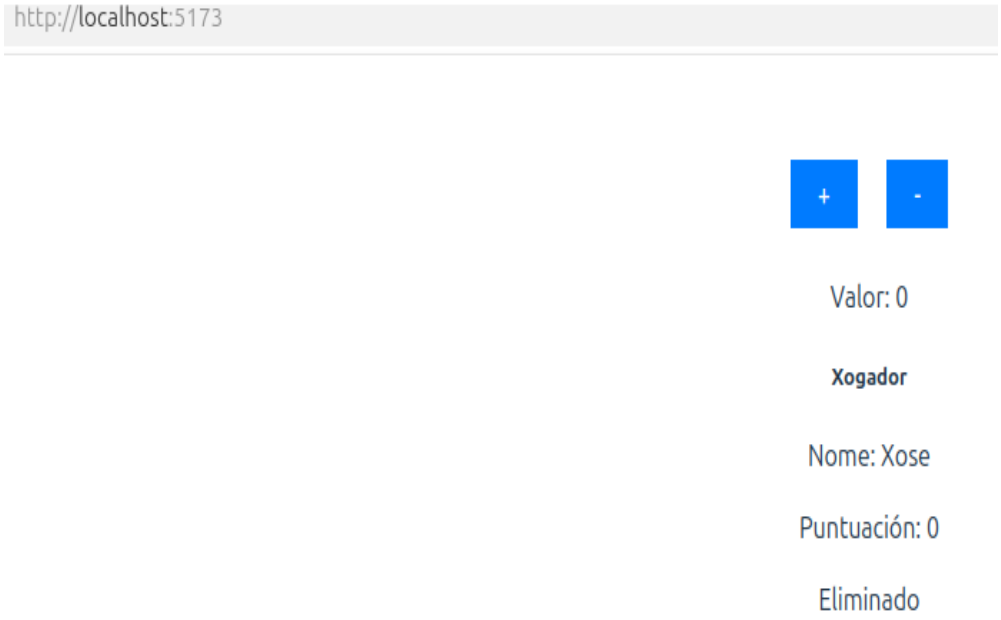
})

function aumentar() {
  contador.value++
  xogador.puntuacion++
}
function restar(){
  contador.value--
  xogador.puntuacion--
}

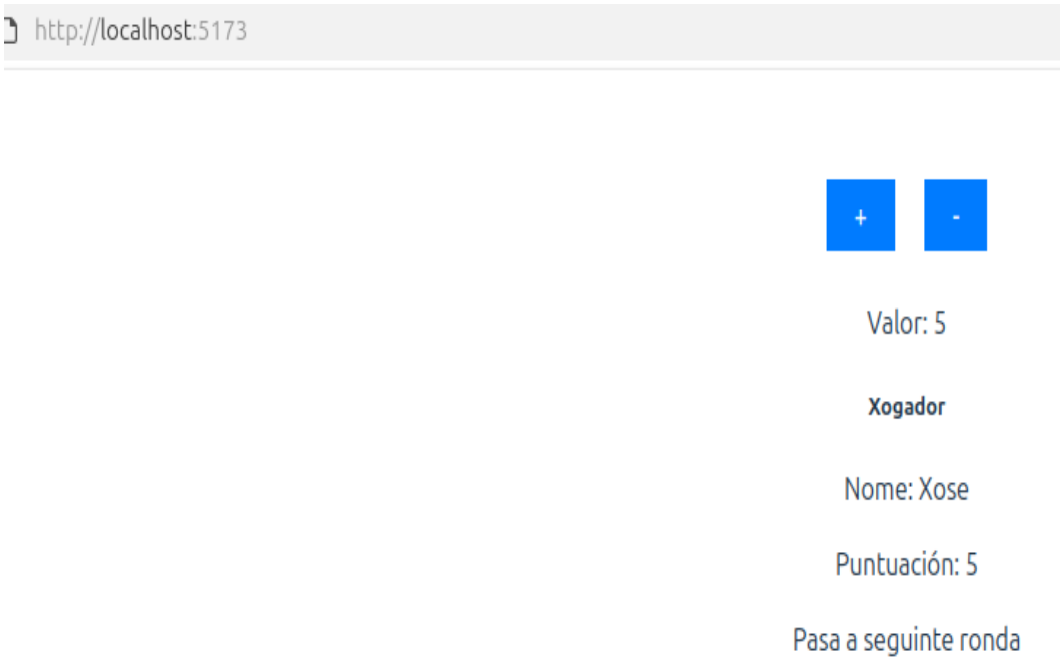
const estadoxogador = computed (()=> {
  if ( xogador.puntuacion >= 5) return "Pasa a seguinte ronda"
  if ( xogador.puntuacion < 5) return "Eliminado"
})


</script>
```

O resultado antes:



e despois de chegar ao valor de puntuación 5.



A.2. Ciclo de vida dun compoñente. Directivas

En ***Vue***, cada compoñente pasa por distintas fases desde que se crea ata que desaparece da pantalla. Esta fases forman o chamado **ciclo de vida do compoñente.**

As fases máis importantes son:

- **Creación**: o compoñente é definido e prepara a súa lóxica interna, pero sen mostrarse aínda.
- **Montaxe**: o compoñente xa está visible no DOM do navegador.
- **Actualización**: cando cambian os datos reactivos, o compoñente vólvese a *renderizar* para mostrar os cambios.
- **Desmontaxe**: o compoñentes elimínase do DOM.

Neste tipo de procesos hai un hook moi útil: ***onMounted(),*** que se executa automaticamente cando o **compoñente xa está montado** no DOM.

Nas versións actuais de **Vue**, especialmente usando ***script setup***, moita lóxica inicial pódese executar directamente dentro do *setup(),* polo que algúns programadores xa empregan menos este *hook*.

Sen embargo, ***onMounted()*** segue sendo **necesario** cando **necesitas acceder ao DOM**, **facer peticións as *APIs*** ou **inicializar datos** só despois de que o compoñente estea completamente cargado.

Resumindo, é o mecanismo ideal para:

- **Cargar os datos** dende unha API, por exemplo, dende unha base de datos.
- **Iniciar temporizadores**
- **Executar código** que precise o compoñente cando estea visible na pantalla.

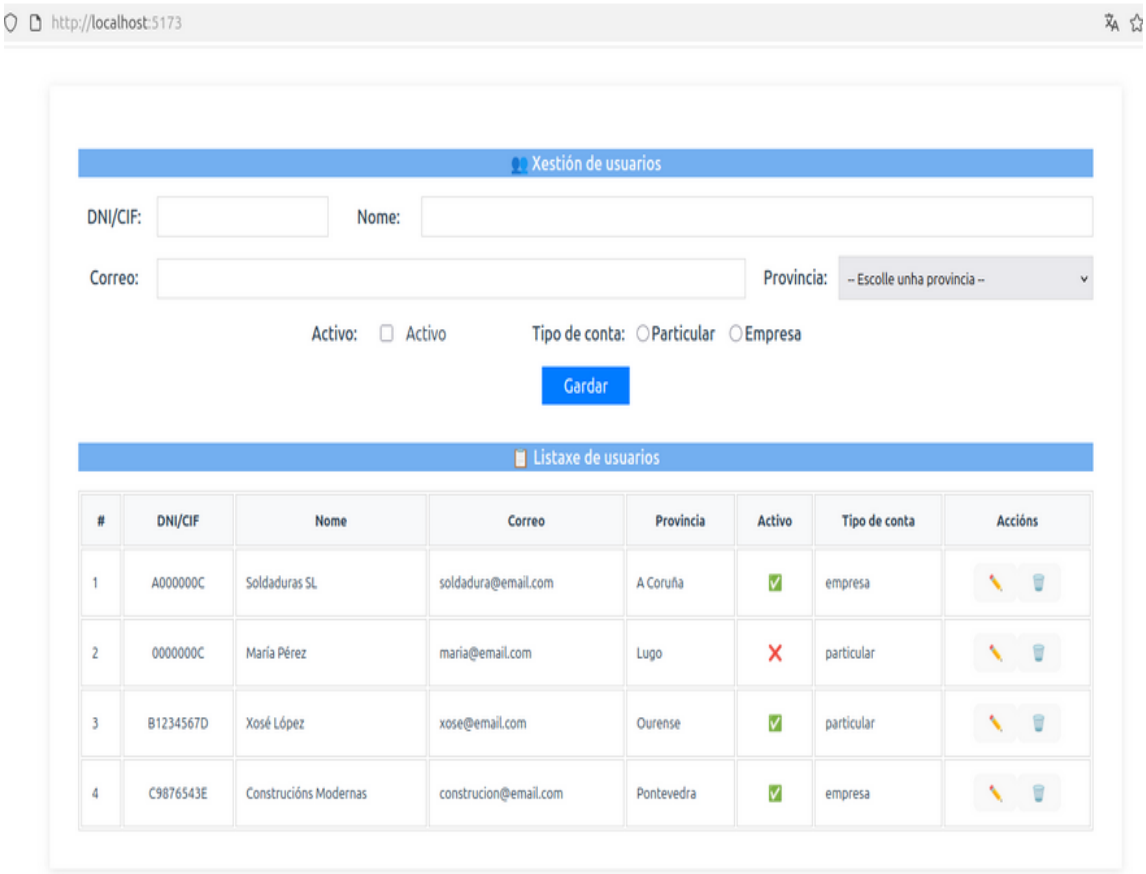
Antes de continuar, no **almacén** deste unidade hai dous arquivos modificados que se corresponden cos anteriores:

- ***XestionUsuarios.vue*** que o cambiamos polo que modificamos antes
- ***App.vue*** que ten lixeiras modificacións de estilo co respecto ao anterior.

Debemos incluílos no entorno substituíndo ao anterior para ir preparando o ***miniproxecto global***. Tras a substitución e lanzamento do proxecto:

npm run dev

debe aparecernos no navegador algo parecido a isto.



Ante de seguir coa explicación da lóxica dos compoñentes, vólvese a insistir que **non entraremos a explicar os estilos CSS** que se propoñen para o deseño da páxina porque queda fora do alcance e obxectivo do curso. De feito, moitos deles son recollidos de outros proxectos realizados na aula, na Internet e, tamén, con algunha modificación con *IA* nalgún aspecto para a súa mellora, aínda que as veces hai que retocalos.

Volvendo os compoñentes, o que temos é un sinxelo formulario é debaixo unha listaxe que nos mostra o que imos engadindo no formulario. Ademais na dereita da táboa temos un lapis que é un botón para **editar un usuario** que se cargue na táboa, e un botón con unha icona dun cubo de lixo para **eliminalo**.

Empecemos botando unha ollada a ***App.vue.***

O arquivo raíz non ten nada especial ou diferente ao da unidade anterior que cargaba *HelloWorld.vue*. Neste caso *script* **importa o compoñente** que quero usar, ***XestionUsuarios.vue*** e na parte do *style* basicamente que apareza no centro ocupando o 80% do ancho e algunhas melloras de estilo adicionais como soporte inicial a dispositivos como *tablets* ou móbiles.

App.vue XestionUsuarios.vue

src > App.vue > ...

```vue
<template>
  <div id="app">
    <!-- Aquí se carga el componente principal DE MOMENTO-->
    <XestionUsuarios />
  </div>
</template>

<script setup>
import XestionUsuarios from './components/XestionUsuarios.vue'
</script>

<style>
/* 👇 Estilos globales mínimos */
:global(body) {
  margin: 0;               /* quita el margen por defecto del body */
  background: #f6f6f6;     /* color de fondo general */
}

#app {
  max-width: 80vw;
  margin: 0 auto;          /* centra horizontalmente */
  padding-top: 1rem;       /* 🔹 margen superior pequeño */
  padding-bottom: 1rem;    /* 🔹 espacio para el futuro footer */
  min-height: 100vh;       /* ocupa toda la altura de la pantalla */
  display: flex;
  flex-direction: column;
  justify-content: flex-start; /* el contenido empieza arriba */
}

</style>
```

Este é o que menos tocaremos ao longo do proxecto. De feito só o tocaremos un par de veces máis e con modificacións mínimas nas unidades posteriores pero que afectarán de gran maneira ao resto de proxecto tal como pode sospeitar deste arquivo chave.

O que ten conceptos substanciais para entender o funcionamento de **Vue** é o *XestionUsuarios.vue.* Pola súa lonxitude iremos por partes:

A. *Template*

Contén un **formulario** e debaixo unha **táboa** onde carga os datos que imos introducindo no formulario.

A seguinte imaxe sería a parte correspondente ao *form:*



```
<template>
  <div class="xestion-usuarios">
    <h4>👥 Xestión de usuarios</h4>
    <form @submit.prevent="gardarUsuario">
      <div class="fila">
        <div class="campo campo-dni">
          <label>DNI/CIF:</label>
          <input v-model="novoUsuario.dni" type="text" required style="text-align: center;" />
        </div>
        <div class="campo campo-nome">
          <label>Nome:</label>
          <input v-model="novoUsuario.nome" type="text" required />
        </div>
      </div>
      <div class="fila">
        <div class="campo campo-correo">
          <label>Correo:</label>
          <input v-model="novoUsuario.correo" type="email" required />
        </div>
        <div class="campo campo-provincia">
          <label>Provincia:</label>
          <select v-model="novoUsuario.provincia">
            <option value="">-- Escolle unha provincia --</option>
            <option>A Coruña</option>
            <option>Lugo</option>
            <option>Ourense</option>
            <option>Pontevedra</option>
          </select>
        </div>
      </div>
      <div class="fila fila-centrada">
        <div class="campo inline-activo">
          <label>Activo:</label>
          <div class="inline-control">
            <input v-model="novoUsuario.activo" type="checkbox" />
            <span>Activo</span>
          </div>
        </div>
        <div class="campo inline-cuenta">
          <label>Tipo de conta:</label>
          <div class="inline-control radios">
            <label>
              <input v-model="novoUsuario.tipoCuenta" type="radio" value="particular" />
              <span>Particular</span>
            </label>
            <label>
              <input v-model="novoUsuario.tipoCuenta" type="radio" value="empresa" />
              <span>Empresa</span>
            </label>
          </div>
        </div>
      </div>
      <button type="submit" class="btn-guardar" :disabled="novoUsuario.dni === '' || novoUsuario.nome === ''">
        Gardar
      </button>
    </form>
    <h4>📋 Listaxe de usuarios</h4>
    <table v-if="usuarios.length > 0">
```

E seguinte e a parte da *táboa:*

```
<h4>📋 Listaxe de usuarios</h4>
  <table v-if="usuarios.length > 0">
    <thead>
      <tr>
        <th>#</th>
        <th>DNI/CIF</th>
        <th>Nome</th>
        <th>Correo</th>
        <th>Provincia</th>
        <th>Activo</th>
        <th>Tipo de conta</th>
        <th>Accións</th>
      </tr>
    </thead>
    <tbody>
      <tr v-for="(u, index) in usuarios" :key="index">
        <td>{{ index + 1 }}</td>
        <td style="text-align: center;">{{ u.dni }}</td>
        <td>{{ u.nome }}</td>
        <td>{{ u.correo }}</td>
        <td>{{ u.provincia }}</td>
        <td style="text-align: center;">{{ u.activo ? "✅" : "❌" }}</td>
        <td>{{ u.tipoCuenta }}</td>
        <td style="text-align: center;">
          <button @click="editarUsuario(index)" title="Editar">✏️</button>
          <button @click="eliminarUsuario(index)" title="Eliminar">🗑️</button>
        </td>
      </tr>
    </tbody>
/table>

  <p v-else>Non hai usuarios cargados.</p>
</div>
/template>
```

Chegados a este punto aparece outro dos conceptos clave na programación co *framework* **Vue**, que son as **directivas**, marcadas elas con subliñado vermello. Vémolas:

- ***v-model:*** vincula un **dato do compoñente que é unha variable da zona do *script*,** cun **elemento dun formulario (*input, select, textarea*, etc.)** de forma **reactiva e bidireccional**.

Isto quere dicir que, **se cambias o valor no campo do formulario**, a variable tamén cambia, **e se modificas a variable no código**, o formulario actualízase automaticamente.

Neste caso temos un modelo que chamamos *novoUsuario* coas súas variables: nome, correo, activo,... que aparece logo na declaración de datos do *script*.

- ***v-if/v-else/ v-else-if***: permite **mostrar u ocultar** partes del *template* **segundo una condición lóxica, e dicir, quita e pon elementos do DOM**.

Existe un similar, **v-show** solo oculta usando CSS con ***display:none,*** é más eficiente se cambias moito entre mostrar/ocultar.

- ***v-bind*** ou simplemente " **:** ", que afecta ao botón e que permite **ligar de forma reactiva un atributo HTML cun dato do compoñente**.

É dicir, calquera atributo dun elemento (*placeholder, disabled, class, title, src, etc.*) pode "tomar" o seu valor dunha variable da zona do *script*. O comportamento é exactamente o mesmo: **se a variable cambia, o atributo actualízase** no navegador automaticamente. Neste caso mentres non engada un *nome* e un *dni* o botón está deshabilitado. Un exemplo,

```
<td :style="{ color: u.edad >= 18 ? 'blue' : 'orange', fontWeight: 'bold' }"> {{ u.edad }} </td>
```

Si o usuario é maior de idade cor azul, senón cor laranxa e negrita

- ***v-for:*** serve para **amosar unha lista de elementos**, repetindo unha parte do *template* (por exemplo, filas dunha táboa ou rexistros do usuario) por cada elemento dun *array* ou colección.

  O atributo ***:key*** é moi **importante para que Vue identifique cada elemento de maneira única** e xestione mellor as actualizacións cando se recorre con for.

Existen varias directivas máis, pero por citar algunha das sinxelas, que non a usamos neste código.

- ***v-on:*** que sustituiría a @ no botón: *v-on:click="engadirUsuario"*
- ***v-text:*** substitúe a **{{ }} :** * *
- ***v-html:*** insire HTML (coidado con seguridade):

  * *

Existe outras máis complexas, algunhas delas verémolas nas seguintes unidades.

En canto o resto do código resto xa é sintaxes de html e css.

B. *Script.*

A parte máis interesante:

Na primeira imaxe vemos a **declaración de variables** e o ***hooked onMounted***

Fixádevos que unha variable, o *array usuarios,* esta declarada como ***ref*** que xa se indicou que se fai para **declarar variables reactivas de tipos de datos primitivos** ou non moi complexos.

A segunda, xa **é una estrutura de datos,** *novoUsuario*, e polo tanto declarámola como ***reactive.*** Ademais é o *"nexo de unión"* ca zona de *template*, xa que **coincide** ca directiva ***v-model.***

```
<script setup>
/// Zona de declaracións
import { ref, reactive, onMounted } from 'vue'

const usuarios = ref([])  //almacena la lista de usuarios e os seus cambios

const novoUsuario = reactive({
  dni: "",
  nome: "",
  correo: "",
  provincia: "",
  activo: false,
  tipoCuenta: ""
})

/// Zona de ciclo de vida

onMounted(() => {       //sempre se cargan estos usuarios de exemplo ao iniciar o componente
  usuarios.value = [
    { dni: "A000000C", nome: "Soldaduras SL", correo: "soldadura@email.com", provincia: "A Coruña", activo: true, tipoCuenta: "empre
    { dni: "0000000C", nome: "María Pérez", correo: "maria@email.com", provincia: "Lugo", activo: false, tipoCuenta: "particular" },
    { dni: "B1234567D", nome: "Xosé López", correo: "xose@email.com", provincia: "Ourense", activo: true, tipoCuenta: "particular" }
    { dni: "C9876543E", nome: "Construcións Modernas", correo: "construcion@email.com", provincia: "Pontevedra", activo: true, tipoC
  ]
})
```

Calquera cambio na zona de *script* que afecta a *novoUsuario* verase reflexada no *template*.

A segundo parte é o ***hook onMounted,*** se **executa cando carga o compoñente.** Neste caso **carga datos** no *array* usuarios declarada na zona anterior cando se monta a páxina.

O ***v-for*** do ***template*** recorre a listaxe de usuarios e cárgaos na táboa, tal como se mostra

Na seguinte imaxe ten que ver cas funcións ou métodos que "fan algo".

```
/// Zona de métodos ou funcións

function gardarUsuario() {
  usuarios.value.push({ ...novoUsuario })  //engade o novo usuario á lista (copia do obxecto)
  Object.assign(novoUsuario, { nome: "", correo: "", idade: null, provincia: "", activo: false, sexo: "" }) //reinicia o formulario
}

function eliminarUsuario(index) {
  usuarios.value.splice(index, 1);   //elimina o usuario da lista
}

function editarUsuario(index) {
  const usuario = usuarios.value[index];   //carga os datos do usuario elixido no formulario
  Object.assign(novoUsuario, usuario);  // carga os datos do usuario no formulario recorda v-model do formulario é novoUsuario
  usuarios.value.splice(index, 1);   //elimina o usuario da lista para que ao gardar se engada de novo cas modificacións
    }

</script>
```

A función ***gardarUsuario,*** o que fai mediante o método *push*, e cargar un *novoUsuario* no *array* usuarios, cando pulsamos o botón Gardar Usuario na zona do *template*.

```
<form @submit.prevent="gardarUsuario">
  <div class="fila">
    <div class="campo campo-dni">
      <label>DNI/CIF:</label>
      <input v-model="novoUsuario.dni" type="text"
```

A función ***eliminarUsuario*** a través da clave *index* da táboa, elimina un usuario do array mediante a función *splice* que coincida co mencionado *index.*

Finalmente ***editarUsuario,*** carga na variable usuario elixido co valor de *index* e asínao a *novoUsuario,* e como *novoUsuario* é una variable reactiva cárgao a súa vez no template, é dicir, no formulario. Recordemos unha vez máis, o ***v-model*** *novoUsuario* é o "nexo de unión" entre a zona de JS e o *template.*

```
<td style="text-align: center;">
  <button @click="editarUsuario(index)" title="Editar">✏️</button>
  <button @click="eliminarUsuario(index)" title="Eliminar">🗑️</button>
</td>
```

A zona de *style*, como dixemos, no é tema deste curso de iniciación.

Para ir finalizando con esta unidade, falamos do ***hook onMounted(),*** pero non é o único, pero sí o máis usado. Na táboa seguinte móstranse outros.

Hook	Momento do Ciclo	Cando se usa normalmente
onBeforeMounted()	Xusto **antes de montar** o compoñente no DOM	Preparar algo antes de que o compoñente se vexa na pantalla
onMounted()	Cando o compoñentes **xa está montado no DOM**	Cargar datos, facer peticións, iniciar temporizadores ou librarías que necesitan o DOM
onBeforeUpdate()	Antes de que o compoñente se **actualice** por un cambio reactivo.	Ver o valor antigo antes de que se renderice de novo ou realizar accións previas
onUpdate()	Despois de que se **actualice o DOM**	Realizar accións tras un cambio visual (animacións, scroll, etc.)
onBeforeUnmount()	Antes de que o compoñente se **desmonte (destruír)**	Limpar temporizadores, listeners, ou conexións abertas
onUnmounted()	Cando o compoñente xa foi **eliminado do DOM**	Confirmar que todo foi liberado correctamente
onActivated()	Cada vez que o compoñente *"volta á vida"* despois de ter estado gardado en caché.	Recargar datos se é necesario. Reiniciar contadores, animacións ou estados que só teñen sentido cando o compoñente está visible.
onDeactivated()	Cando o compoñente pasa a estar oculto pero **non destruído**, porque segue gardado	Pausa temporizadores, garda estados o cancela listeners cando paso a outra páxina.

A.3. Resumo

- Comprendemos como *Vue* **actualiza automaticamente** a interfaz grazas á **reactividade.**
- Aprendemos a usar ***ref, reactive e computed***
- Coñecemos as **fases do ciclo de vida** dun compoñente e o *hook* ***onMounted.***
- Aplicamos todo nun compoñente práctico cun formulario e unha táboa para mostrar os datos.